

Práctica 6: Módulos aritméticos y lógicos

Objetivos

Empleo de módulos aritméticos y lógicos.

Se trabajará en los circuitos:

- **Ej 6.1: Comple 2.**
- **Ej 6.2: SumRes 4b.**
- **Bit signo (4b).**
- **Ej 6.4: CompBits1.**

Librería Aritmética

La librería Aritmética de Logisim proporciona módulos aritméticos y lógicos. Se trata de circuitos combinacionales con funciones lógicas específicas y de cálculo aritmético.

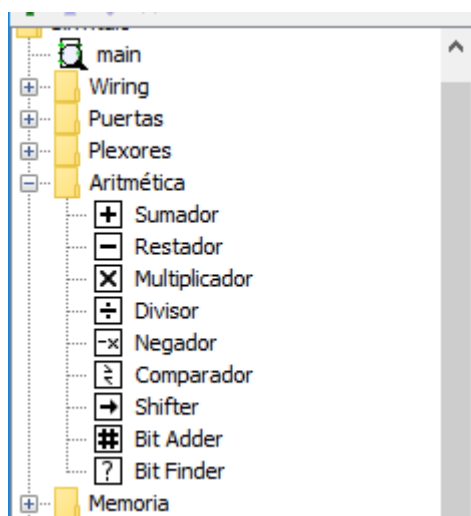


Figura 6.1: Componentes de la librería Aritmética.

Los módulos proporcionados por la librería son:

- **Sumador, Restador, Multiplicador y Divisor.** Realizan las operaciones aritméticas indicadas.
- **Negador.** Cambia el signo de su entrada empleando la representación en complemento a 2.
- **Comparador.** Comparador lógico de dos entradas con tres salidas correspondientes al resultado de la comparación (mayor que '>', igual que '=', menor que '<').
- **Sumador bit a bit (Bit Adder).** Indica el número de bits activos (a nivel alto) en su entrada.
- **Bit Finder.** Permite obtener el índice del bit indicado por uno de sus posibles cuatro criterios de búsqueda: 0/1 de menor orden, 0/1 de mayor orden.

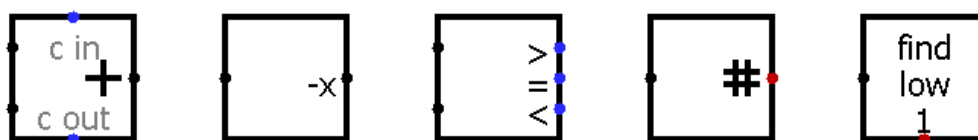


Figura 6.2: Símbolos de Logisim para los componentes de la librería Aritmética.

Ejercicio 6.1: Obtención alternativa de complemento a 2

- Analiza el circuito "**Ej 6.1: Comple2**".
- Observa cómo obtener el complemento a 2 de un número de 8 bits mediante la definición:
$$C2 = C1 + 1$$

Es decir, el complemento a 2 se puede obtener mediante el complemento de bits (C1) y suma de 1. En este caso emplea el módulo sumador proporcionado en Logisim.
- Compara el resultado con el obtenido mediante el módulo negador.

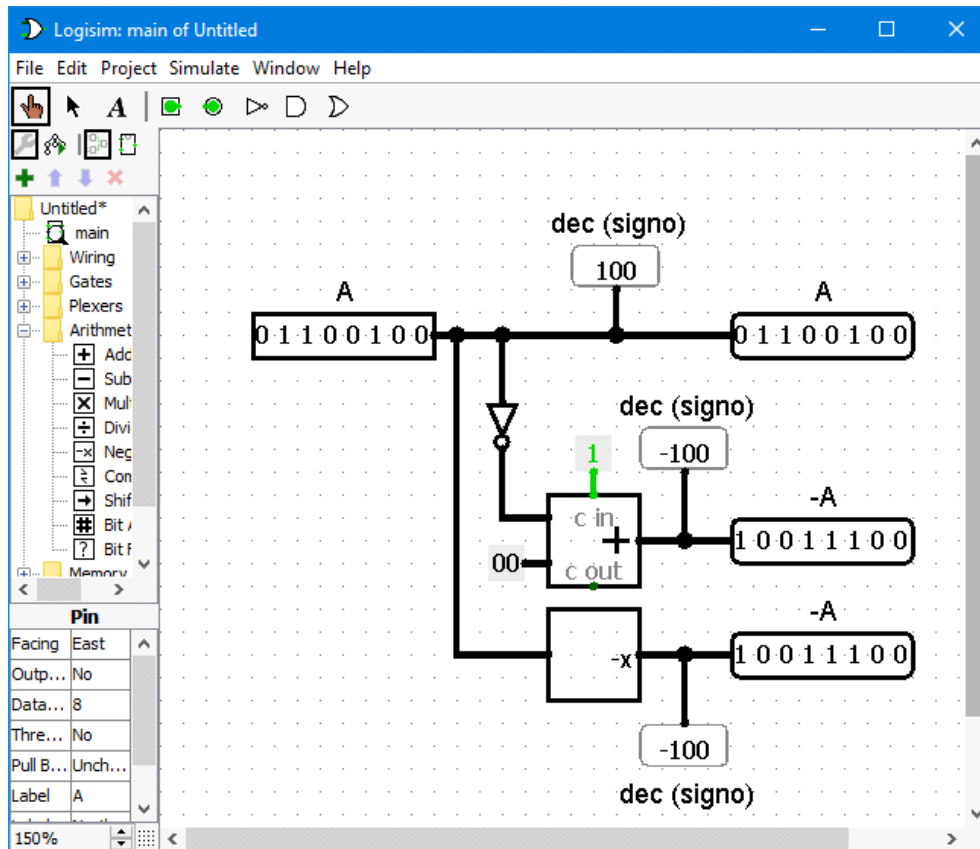


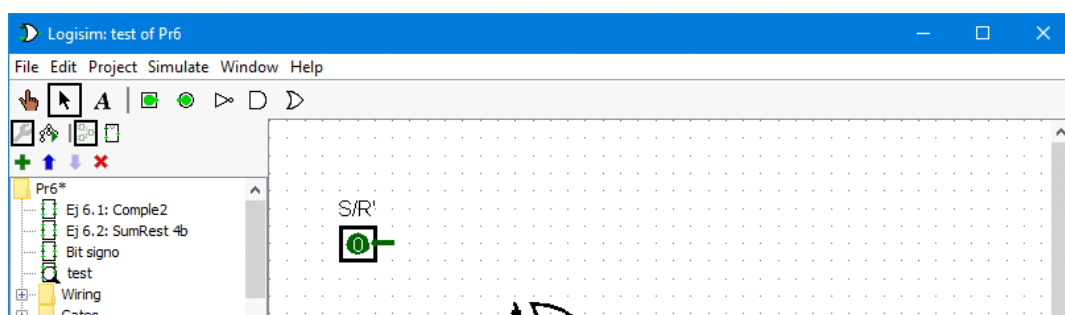
Figura 6.3: Circuito a realizar en el ejercicio 6.1.

Modulo Sumador (restador)

Una de las ventajas del empleo de la codificación de números enteros en complemento a 2 es que tanto las operaciones de suma como las de resta se pueden realizar mediante la suma, ya que la resta se obtiene como la suma del sustraendo cambiado de signo. Además, en este caso no es preciso sumar el acarreo de salida para obtener el resultado correcto de la operación. Por este motivo es habitual implementar en las unidades aritmético lógicas (ALU) un módulo sumador restador que permite implementar tanto la suma como la resta de sus operandos. En estos módulos la resta se obtiene mediante el cálculo del complemento a dos del sustraendo. Para ello aplica la operación de complemento y se suma 1. Utilizando un sumador convencional este resultado se obtiene mediante complemento con una puerta XOR y suma de un bit de acarreo.

Ejercicio 6.2: Implementación de un sumador restador

- Completa el circuito "Ej 6.2: SumRest 4b" para implementar un sumador restador de 4 bits empleando el módulo aritmético de la suma.
- Controla el tipo de operación mediante una señal S/R' que indica el tipo de operación (0=suma, 1=resta).
- Obtén el complemento a 2 mediante inversión del sustraendo y suma de 1 (señal de control de operación). Para la inversión utiliza una batería de puertas XOR (puerta XOR de 4 bits de datos).
- Añade sondas de prueba para monitorear el valor de los operandos y el resultado.



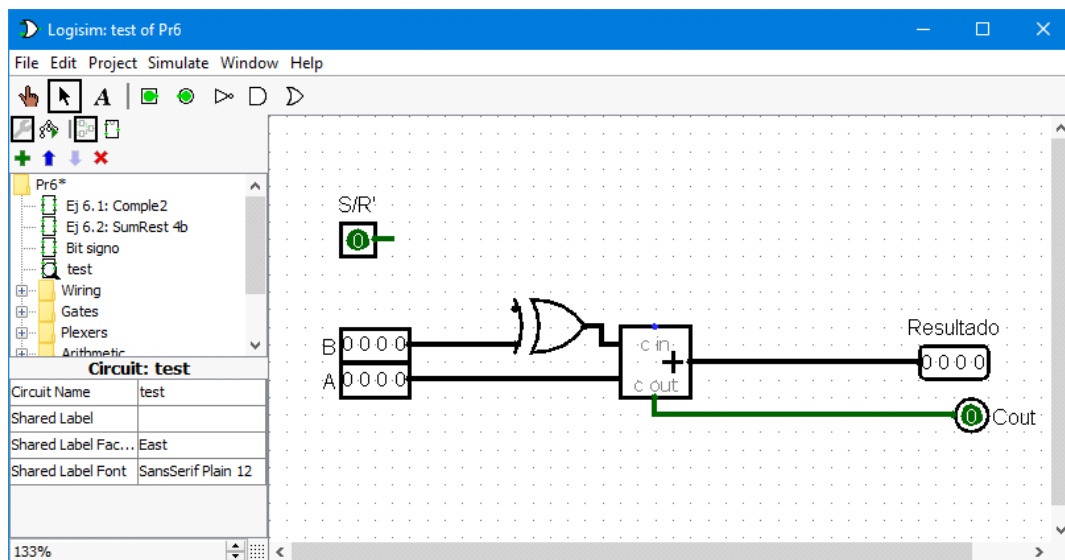


Figura 6.4: Esquema de circuito empleado como pista del ejercicio 6.2.

Detección de desbordamiento

El desbordamiento u *overflow* se produce cuando el resultado de una operación aritmética no se puede representar en el número de bits disponible. En el circuito anterior se produce un *overflow* cuando el resultado de la operación A+B o A-B no puede representarse con 4 bits. Por tanto, el resultado mostrado por el circuito es incorrecto.

Para detectar la condición de *overflow* se tiene en cuenta que este solo se puede producir si el signo de los operandos de entrada en el módulo sumador, es idéntico. Esto es, son ambos positivos o negativos, ya que solo en este caso la magnitud del resultado excede la de los operandos. En esta situación, el desbordamiento se detecta cuando el signo del resultado no coincide con el de los operandos. En resumen, primero se compara el signo de los operandos, cuando es idéntico y no coincide con el del resultado, se ha producido *overflow*.

El bit de signo de un valor binario representado en complemento a dos coincide con el bit más significativo. Para obtener dicho bit en un circuito con Logisim se puede emplear bien un separador de bits o un módulo selector de bits.

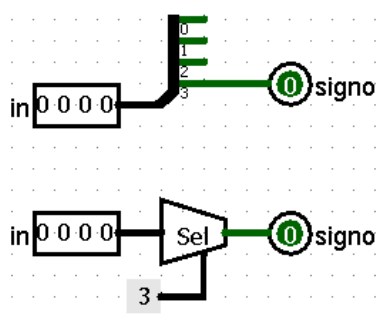


Figura 6.5: Circuitos alternativos para la obtención del bit de signo (bit más significativo) en una señal de entrada.

★ **NOTA:** La condición de igualdad entre dos bits se puede realizar de un modo sencillo con la puerta XNOR de dos entradas que produce un 1 siempre que sus entradas coinciden. Por el contrario la desigualdad se obtendría con la puerta XOR.

Ejercicio 6.3: Sumador restador con detección de *overflow*

- Completa el subcircuito "Bit signo (4b)" que obtiene el bit de signo de una entrada de 4 bits.
- Completa el circuito "Ej 6.2: SumRest 4b" con los elementos necesarios para añadir un bit de detección de *overflow*.
- La condición de *overflow* se activa cuando el signo de los operandos es idéntico, pero distinto

del signo del resultado.

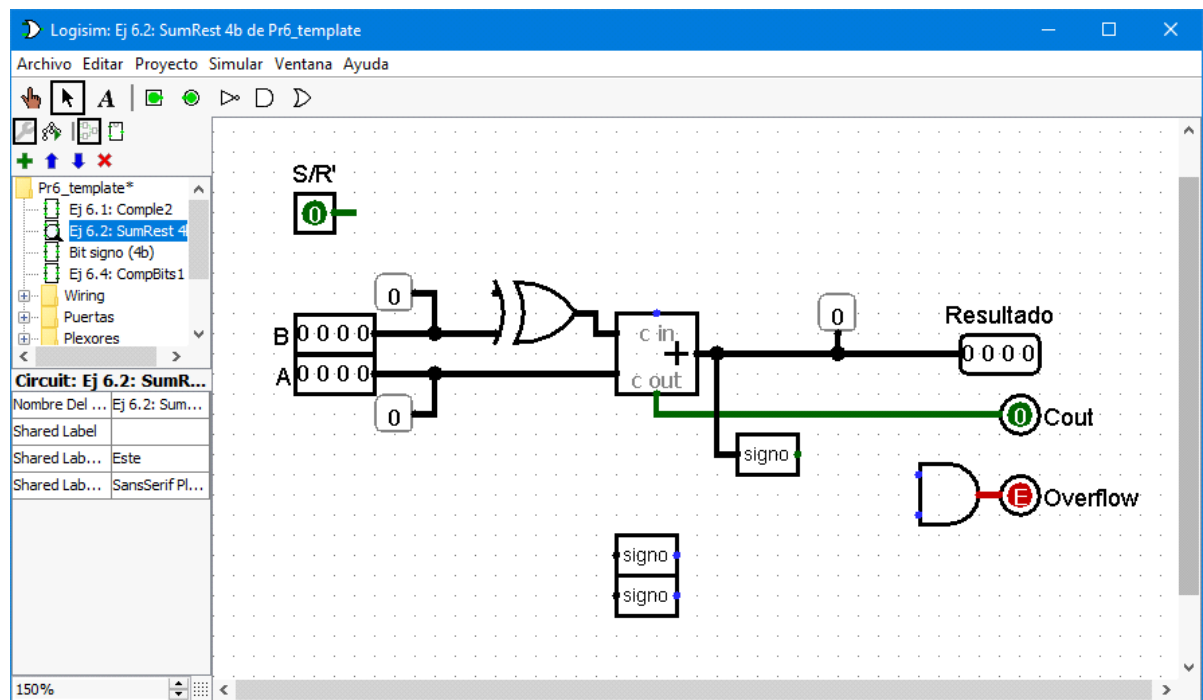


Figura 6.6: Circuito sumador restador con detección de desbordamiento.

Módulos lógicos

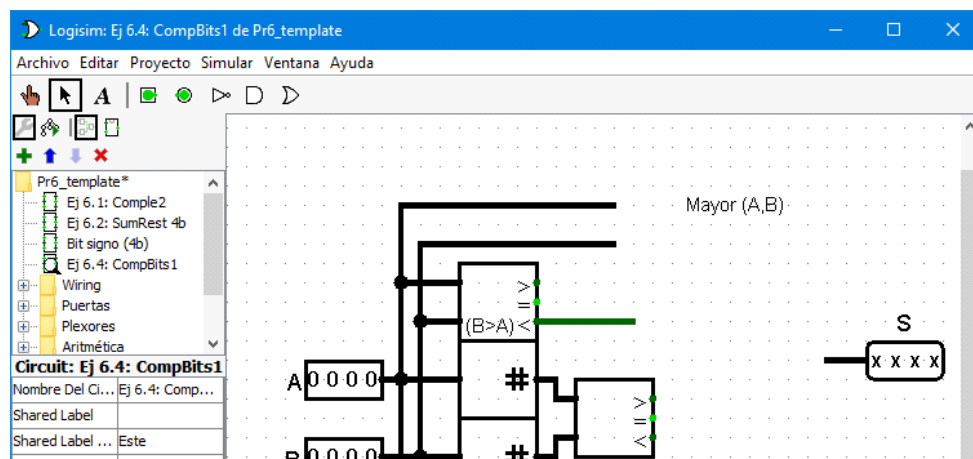
Logisim proporciona dos módulos lógicos que son muy versátiles en los sistemas digitales:

- **Comparador,**
- **Sumador bit a bit (Bit adder).**

Estos módulos en conjunción con los multiplexores permiten implementar sistemas digitales muy interesantes.

Ejercicio 6.4: Comparador de número de bits activos

- Completa el circuito "Ej 6.4: CompBits1" con dos entradas de 4 bits y cuya salida es la entrada con más bits activos. En caso de que el número de bit activos sea igual en ambas entradas, la salida debe coincidir con la mayor de ellas.
 - Utiliza el bit adder para obtener el número de bits activos de cada entrada.
 - Utiliza el comparador para obtener el número mayor de bits activos.
 - Utiliza un MUX 2x1 de 4 bits para obtener la entrada con mayor número de bits activos.
 - Utiliza un MUX 2x1 de 4 bits para obtener la entrada mayor, cuando el número de bits activos es igual.
 - Utiliza un MUX 2x1 de 4 bits para decidir qué salida obtener dependiendo de que el número de bits activos en las entradas sea igual o distinto.



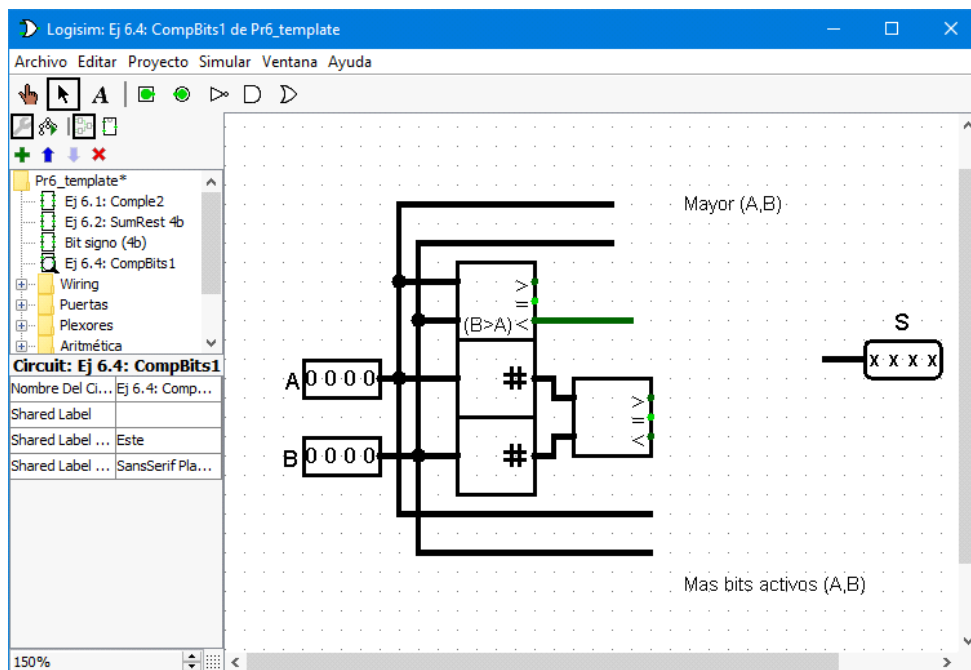


Figura 6.7: Esquema de circuito empleado como pista del ejercicio 6.4.

Entregable

- Salva el proyecto donde has ido realizando los ejercicios propuestos y súbelos a la tarea asociada a esta práctica. Nombra al fichero "<GrupoPr_Apellido_InicialNombre>_Pr6.circ".